Chapter 4 Exercises

Use the five-stage single-issue subset without architectural branch delay slots. Show stages, cycles, bypass sources, and reasons for stalls or flushes.

CH04-Q01 The single-cycle critical path

Component	Delay in ps
PC clock-to-Q	30
Instruction memory	200
Register read	120
ALU	180
Data memory	250
Result mux	20
Register setup	40

A load follows all components in that order. In this simplified question the R-format path bypasses both data memory and the memory-to-register mux.

1. Find the load-limited clock period.
2. Find the difference between the two paths.
3. Explain the effect of reducing only R-format delay by 100 ps.
4. What happens if data-memory delay falls by 80 ps and the load remains critical?

CH04-Q02 Pipeline timing

Seven instructions execute without hazards in five perfectly overlapping stages.

1. Draw each instruction's stages by cycle.
2. Find the final WB cycle.
3. Give total cycles and finite-run average IPC. Separately fix a 200 ps pipeline cycle and an unpipelined cost of 1000 ps per instruction, ignoring added latch overhead. Find the elapsed-time speedup for these same seven instructions.
4. If each stage takes 200 ps, find single-instruction latency and total completion time.

Chapter 4 Exercises (continued)

CH04-Q03 Forwarding

```
I1: add $8, $9, $10
I2: sub $11, $8, $12
I3: and $13, $11, $8
```

1. Draw the schedule with one cycle per stage and full ALU forwarding.
2. Identify the source of I2's `$8`.
3. Identify both sources needed by I3 in EX.
4. Explain which stale values can be consumed if forwarding is omitted.

CH04-Q04 Load-use dependence

```
I1: lw  $2, 20($1)
I2: and $4, $2, $5
I3: or  $6, $1, $7
```

1. Derive the stall condition.
2. State the actions on PC, IF/ID, and ID/EX.
3. Draw the schedule with the bubble.
4. Replace I2 with `sw $2,24($1)`. Explain the source-role difference and how a MEM-stage store-data bypass can change the stall requirement.

CH04-Q05 Branch prediction

A two-bit saturating counter has states SN, WN, WT, ST. T increments toward ST; N decrements toward SN. WT and ST predict T. Starting at WN, observe `T,T,N,T,N,N,N`.

1. Trace predictions and states.
2. Compute misses and accuracy.
3. Assume correct target supply and two squashed slots for every direction error, irrespective of actual outcome. Find wrong-path slots.
4. Why does direction accuracy alone not determine pipeline penalty?

CH04-Q06 Precise exceptions

I4 faults with arithmetic overflow in EX. In the same cycle, I1 is already committed, I2 is in WB, I3 is in MEM without a fault, I5 is in ID, and I6 is in IF. All older instructions can complete normally.

Chapter 4 Exercises (continued)

1. Which architectural writes must remain?
2. Which writes must be prevented?
3. What information must the handler obtain?
4. How does a ROB preserve this state in an out-of-order core?

CH04-Q07 Hazard logic

1. Write the load-use condition using `ID/EX.MemRead`, its `RegisterRt`, and the IF/ID source fields.
2. Why exclude `$zero` as a producer?
3. How does a source-use enable prevent a false stall when a load's `rt` is only a destination?
4. If EX/MEM and MEM/WB both match a source, which valid produced value wins?

CH04-Q08 Multiple issue

A 10, 000-instruction program runs at 2 GHz. In 60% of cycles the processor issues four instructions, in 30% it issues two, and in 10% it issues one. Ignore startup, drain, and separately unmodeled stalls.

1. Compute cycle-weighted IPC and approximate cycles.
2. Compute execution time.
3. Compare with a 2.4 GHz machine sustaining two instructions per cycle.
4. Explain what the model omits about prediction, misses, and port conflicts.

CH04-Q09 Multicycle FSM and clock

R-type/load/store/branch-or-jump take 4/5/4/3 cycles. The mix is 45% R-type, 20% branch/jump, 20% load, 15% store. Controller cycles take 300 ps.

1. Find weighted CPI.
2. Find time for 10^9 instructions.
3. Compare a single-cycle 800 ps, CPI 1 baseline.
4. With fixed mix/state paths, what maximum multicycle period matches it?

Chapter 4 Exercises (continued)

CH04-Q10 Branch slots, flush, and precise exceptions

1. With 20% branches, 60% taken, always-not-taken prediction, EX resolution, and two-slot no-delay-slot recovery, find misprediction cycles per instruction.
2. Add one load-use bubble for 10% of instructions and no other stalls. Find CPI.
3. A classic MIPS slot reduces post-slot wrong-path flushes to one. Why must dynamic instruction count also be recalculated?
4. At T, I4 faults in EX, I2 is in WB, I3 in MEM, I5 in ID, I6 in IF, and I1 already committed. Give exception capture, older drain, younger flush, and handler redirect actions by cycle.

CH04-Q11 Reconstruct muxes and next PC

Use the no-delay-slot single-cycle subset. Each part independently starts at PC=0x00400020, s0=0x1000, s1=7, s2=5, t2=0x12345678, Mem[0x100c]=0xabcdef01; other registers are zero. Accesses are aligned and no overflow occurs.

1. Decode 0x02324021, 0x8e09000c, 0xae0afffc. Give RegDst, ALUSrc, MemtoReg, WriteReg, ALU inputs/result, and committed memory/register effects.
2. For 0x1232fffe, find PC4, BT, Zero, nextPC. Repeat with s1=5.
3. For j field zero at PC=0x0ffffffc, find nextPC and explain the use of PC+4's upper bits.
4. If sw incorrectly selects ALUSrc 0, find the address. If address is correct but memory data is the sign-extended immediate, find the stored value. Identify each faulty wire.

CH04-Q12 Reconstruct timing from an older value and a load

Use the four-instruction numeric trace, initial state, and five-stage model in the slides. WB writes in the first half-cycle; ID reads in the second half.

Initially `$1=0x1000`, `$6=0x30`, `$7=0x03`, `$5=0x0f`, `$9=0x80`, and `Mem[0x1014]=0x3c`; other relevant registers are zero. There are no cache misses, branches, structural conflicts, misaligned accesses, or exceptions.

```
I1  add $2,$6,$7
I2  lw  $2,20($1)
I3  and $4,$2,$5
I4  or  $8,$4,$9
```

Chapter 4 Exercises (continued)

1. With EX/MEM and MEM/WB bypasses to EX, list every instruction's IF, ID, EX, MEM, and WB cycles, including repeated stage occupancy.
2. Which `$2` value can ID/EX latch at C5's end? Why does I3 still compute correctly? Give the C6/C7 bypass sources, values, and final `$4` and `$8`.
3. Remove all forwarding. Reconstruct the timing and total cycles without changing the register-file convention.
4. Restore forwarding, replace I3 by `and $4,$10,$5`, and initialize `$10=0x55`. Is a load-use stall still needed? Find total cycles and `$8`.

References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study